Timers, Timing and PWM

Introduction to ARM Cortex-M Microprocessors — Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

Why This Chapter Matters

`delay_ms()` has three problems. Its timing changes with the optimizer and the clock. It uses the CPU while it waits. It gives no accurate time base. A hardware timer solves all three. It counts on its own, so it can **schedule** an event, **measure** an event, or **generate** a timed output such as PWM.

After this chapter, you can:

1. Explain how prescaler and reload set a timer's period.
2. Calculate prescaler and reload values.
3. Generate periodic update interrupts.
4. Keep a millisecond time base with SysTick.
5. Choose the alternate function that routes a pin to a timer channel.
6. Calculate PWM frequency and duty cycle.
7. Apply PWM to LED brightness and motor speed.

© Muním Zabidi

By the end of this chapter you will know why these two assignments give a blink that is exact, however busy the CPU is:

```
TIM2->PSC = 15999;   // 16 MHz / 16000 = 1 kHz
TIM2->ARR = 999;     // 1000 ticks = 1 s
```

Questions this code raises, and where this chapter answers them:

Question raised	Answered in
What is PSC?	Timer architecture
Why 15999 and 999?	Timing calculations
Why is it exact?	Why deterministic timing matters
Can it drive a pin directly?	PWM

© Munim Zabidi

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

Three Things Wrong With `delay_ms()`

```
void delay_ms(int n) {
        for (volatile int i = 0;
        i < n * 4000; i++) ;
}
```

1. Its timing changes with the compiler optimisation level and the clock frequency
2. It consumes 100% of the CPU while waiting
3. It cannot be interrupted accurately — any ISR that fires makes the delay longer

A timer is a counter in hardware

It runs independently of the processor, at a rate you configure. It does not care what the CPU is doing, what the optimiser did, or how many interrupts fired. That is what makes it a clock rather than a guess.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 **Timer Architecture**

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

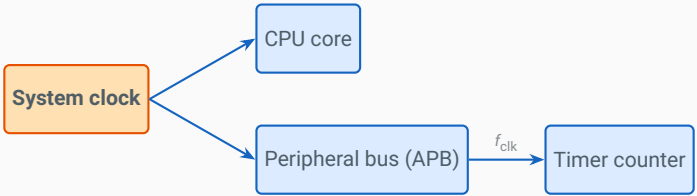
11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

Where Does the Timer Clock Come From?

"The clock" is not one signal. The system clock is divided into several clocks, and they need not run at the same frequency.



A timer counts the clock of the bus it is attached to. Appendix B lists which timer sits on which bus. On this board the default is 16 MHz everywhere, so the numbers agree. They will not agree once the clock tree is changed.

© Muním Zabidi

The Timer Chain and Its Registers

A chain of stages

1. The peripheral clock supplies raw ticks, for example 16 MHz.
2. The prescaler slows them down.
3. The counter counts the slower ticks.
4. The auto-reload register sets how many counts make one cycle.
5. Reaching it gives an update event. The counter restarts at zero.
6. The event can interrupt the CPU or drive another timer function.

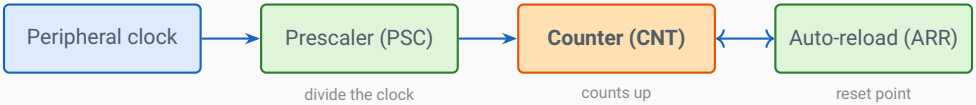
Four registers that matter

- `PSC`: divides the clock by PSC + 1. A value of 0 must mean divide-by-1.
- `CNT`: the current count. Read it to measure elapsed time.
- `ARR`: the final count. At the end the counter wraps to zero.
- `CCRx`: the threshold for capture, output compare and PWM.

Everything else in this chapter is a variation on step 6: what the update event, or a related compare event, is used for.

© Muním Zabidi

Prescaler and Auto-Reload



$$f_{\text{event}} = \frac{f_{\text{CK}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

The $+1$ is because a value of 0 means “divide by 1” — a counter that reloads at 0 still counts one tick.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 **Timing Calculations**

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

A Timing Calculation

Required

A 1 Hz interrupt from a 16 MHz clock.

Choose a prescaler

PSC $=$ 15999 divides 16 MHz down to 1 kHz — a convenient 1 ms tick.

Then the reload value

ARR $=$ 999 gives 1000 ticks $=$ 1 s.

Check: $\dfrac{16{,}000{,}000}{16000 \times 1000} = 1\,\text{Hz}.$

```
TIM2->PSC = 15999;
TIM2->ARR = 999;
```

Why choose PSC first

Both registers are 16-bit, so neither can exceed 65535. Pick the prescaler to give a convenient tick — 1 ms or 1 µs — then the reload value is just the number of ticks you want.

Worked Example: A 1 ms Tick

Design a timer that interrupts every 1 ms from a 16 MHz clock. Use $T_{\text{update}} = \dfrac{(\text{PSC}+1)(\text{ARR}+1)}{f_{\text{clock}}}$.

$$(\text{PSC}+1)(\text{ARR}+1) = f_{\text{clock}}\, T$$
$$= 16 \times 10^{6} \times 10^{-3} = 16{,}000$$

Many pairs work. Pick a convenient tick first. Choose $\text{PSC}+1 = 16$. The timer clock is then 1 MHz: one tick per microsecond.

$$\text{ARR}+1 = \frac{16{,}000}{16} = 1000$$

$$\text{PSC} = 15, \quad \text{ARR} = 999$$

```
TIM2->PSC = 15;
TIM2->ARR = 999;
```

Why a 1 MHz tick is convenient

The counter finishes a cycle after 1000 ticks, so the period is exactly 1 ms. And the `ARR` value matches the period in microseconds directly.

Worked Example: A 0.5 s Blink

For a 500 ms period from 16 MHz:

$$(\text{PSC}+1)(\text{ARR}+1) = 16 \times 10^6 \times 0.5 = 8{,}000{,}000$$

ARR is 16 bits, so it holds at most 65535. The prescaler must take enough of the division to keep ARR in range.

Choose $\text{PSC}+1 = 1600$, so $\text{PSC} = 1599$.

The timer clock is 10 kHz.

$$\text{ARR}+1 = \frac{8{,}000{,}000}{1600} = 5000$$

$$\text{ARR} = 4999$$

Always check the width of ARR

If the value you need does not fit in 16 bits, increase the prescaler. Here both values fit comfortably.

Predict the Period Work it out before you check

A 16 MHz timer clock. Registers are set like this:

```
TIM2->PSC = 799;
TIM2->ARR = 1999;
```

Your prediction

- What is the timer tick frequency?
- How many ticks make one cycle?
- What is the update period?

A 16 MHz timer clock. Registers are set like this:

```
TIM2->PSC = 799;
TIM2->ARR = 1999;
```

Your prediction

- What is the timer tick frequency?
- How many ticks make one cycle?
- What is the update period?

The answer

Tick $= 16\,\text{MHz}/800 = 20\,\text{kHz}$. One cycle $= 2000$ ticks. Period $= 2000/20{,}000 = 0.1\,\text{s}$ (10 Hz). Check: $(800 \times 2000)/16 \times 10^6 = 0.1$.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 **A Periodic Timer Interrupt**

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

A Periodic Timer Interrupt

```
RCC->APB1ENR |= (1U << 0);      // TIM2 clock
TIM2->PSC = 15999;              // 16 MHz -> 1 kHz
TIM2->ARR = 999;                // 1000 ticks -> 1 Hz
TIM2->DIER |= (1U << 0);        // update interrupt enable
TIM2->CR1  |= (1U << 0);        // start counting
NVIC_EnableIRQ(TIM2_IRQn);

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1U << 0)) { // update flag?
        TIM2->SR &= ~(1U << 0); // clear it
        GPIOA->ODR ^= (1U << 5);
    }
}
```

The CPU is now free between interrupts — it can sleep, or do real work, and the LED still blinks at exactly 1 Hz.

Table of Contents

1 Why This Matters
2 Why Software Delays Fail
3 Timer Architecture
4 Timing Calculations
5 A Periodic Timer Interrupt
6 **Why Deterministic Timing Matters**
7 SysTick: The System Time Base
8 Alternate Function Selection
9 Pulse Width Modulation
10 Input Capture and Output Compare
11 Engineering Case Study: Sampling Audio
12 Beamer Extras
13 Summary

Why Deterministic Timing Matters

Once configured, a hardware timer keeps counting at a fixed rate whatever the CPU is doing. Its period depends only on PSC and ARR. A software delay changes whenever the workload changes.

Timing as a property of hardware, not of software

- Several activities can share one processor without one's timing depending on another's workload.
- The CPU stays free to do other work between events.
- An action can happen at a truly uniform rate, not merely "often enough".

TIM2 solved one job: blink this LED at this rate. A program also needs a shared sense of elapsed time. That is a different job with its own hardware: SysTick, next.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 **SysTick: The System Time Base**

8 Alternate Function Selection

9 Pulse Width Modulation

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

- A 24-bit down-counter built into the core, not the vendor's peripherals.
- Present on every Cortex-M, at the same address — so code using it is portable.
- Normally configured for a 1 ms tick.

```
SysTick_Config(SystemCoreClock / 1000);
```

```
volatile uint32_t ticks = 0;

void SysTick_Handler(void)
{
        ticks++;
}

uint32_t millis(void)
{
        return ticks;
}
```

This is what replaces `delay_ms()`

Instead of *waiting*, record when something last happened and check whether enough time has passed. That is the non-blocking pattern Chapter 14 builds a whole system from.

© Muním Zabidi

The Bad `delay_ms()` — For Comparison

```
void delay_ms_bad(int n)
{
        for (volatile int i = 0; i < n * 4000; i++)
        ;
        /* Wrong in three ways: drifts with optimisation level
        and clock frequency, burns 100% CPU, and any ISR that
        fires makes it longer than requested. Do not use this
        in real designs. */
}
```

A Proper `delay_ms()` — Built on SysTick

```c
static volatile uint32_t systick_ms = 0;

void SysTick_Handler(void) { systick_ms++; }

void systick_init(uint32_t core_clock_hz)
{
        SysTick_Config(core_clock_hz / 1000U); // 1 ms tick
}

uint32_t millis(void) { return systick_ms; }

// Blocking, but exact regardless of compiler optimisation:
// it counts real elapsed ticks, not a fixed loop count.
void delay_ms(uint32_t n)
{
        uint32_t start = millis();
        while ((millis() - start) < n) {
                __asm volatile ("wfi");  // sleep between ticks
        }
}
```

An Alternative: `delay_ms()` on a General-Purpose Timer

```
void tim2_delay_init(void)
{
        RCC->APB1ENR |= (1U << 0);  // TIM2 clock enable
        TIM2->PSC = 15999;          // 16 MHz -> 1 kHz
        TIM2->ARR = 0xFFFF;         // free-running
        TIM2->CR1 |= (1U << 0);     // start counting
}

void delay_ms_tim2(uint32_t n)
{
        TIM2->CNT = 0;
        while (TIM2->CNT < n)
        ;
}
```

Useful when SysTick is already claimed by an RTOS, and the mechanism is the same prescaler/auto-reload arithmetic from earlier in this chapter.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 **Alternate Function Selection**

9 Pulse Width Modulation

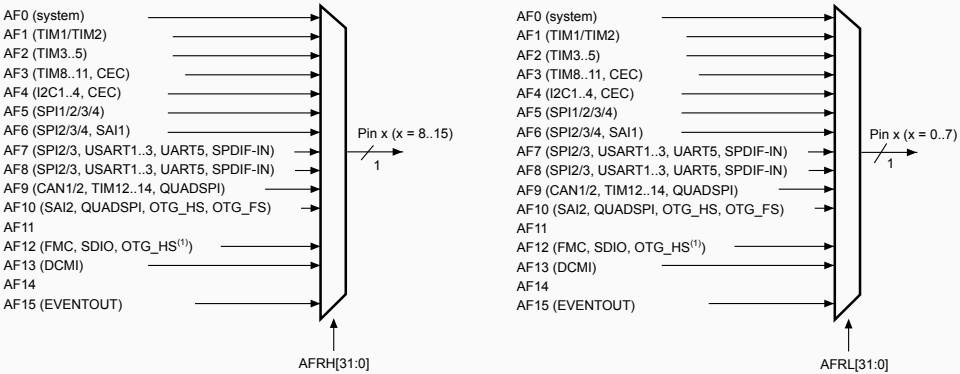
10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary

Routing a Pin to a Peripheral



A pin can serve several peripherals. `MODER` set to `10` selects alternate-function mode; `AFR` then chooses which of up to sixteen functions.

Selecting the Function Number

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Four bits per pin, so `AFR[0]` covers pins 0–7 and `AFR[1]` covers pins 8–15.

For pin $n \geq 8$ the shift is $(n - 8) \times 4$.

```
// PA5 -> TIM2_CH1 is AF1
GPIOA->MODER &= ~(3U << 10);
GPIOA->MODER |=  (2U << 10);

GPIOA->AFR[0] |= (1U << (5*4));
```

The function numbers are in the datasheet, not the reference manual — a distinction that costs people an afternoon.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

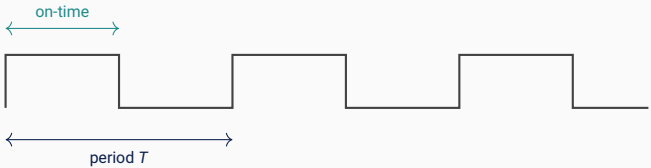
9 **Pulse Width Modulation**

10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 Summary



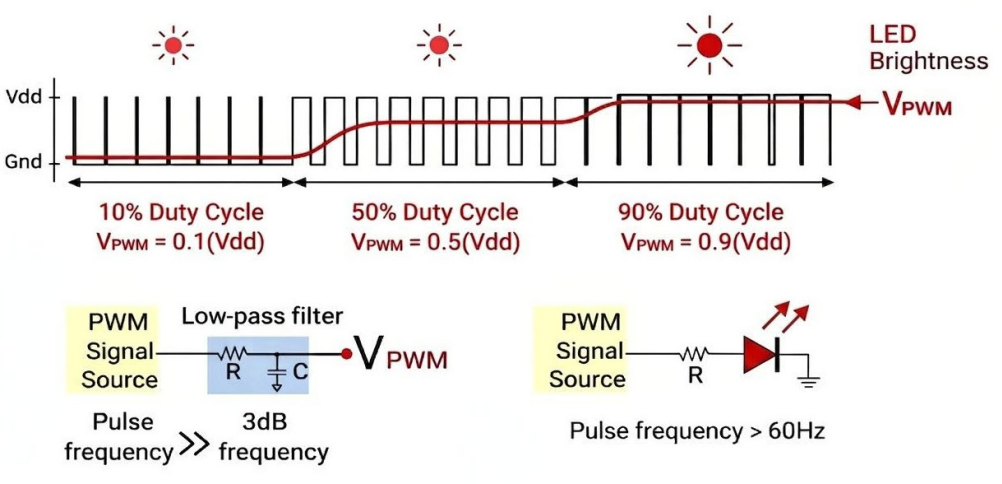
$$\text{duty cycle} = \frac{\text{on-time}}{T} \qquad f_{\text{PWM}} = \frac{f_{\text{CK}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

A digital pin producing an analog effect

The pin is only ever fully on or fully off — but switch it fast enough and an LED looks dimmed, a motor runs slower, a heater delivers less power. The average is what the load responds to.

© Muním Zabidi

LED Brightness Control with PWM



Choosing a PWM Frequency

Load	Guidance
LEDs	Above about 100 Hz avoids visible flicker. 1 kHz is a common choice.
DC motors	Above about 20 kHz puts the switching tone beyond human hearing.
Servos	About 50 Hz, with pulse widths of 1–2 ms. Here the pulse width *is* the command, not an average voltage.

These are conventions, not universal limits. The device's datasheet decides.

Configuring PWM

```
TIM2->PSC  = 15;                // 16 MHz -> 1 MHz
TIM2->ARR  = 999;               // 1 MHz / 1000 = 1 kHz PWM
TIM2->CCR1 = 250;               // 250/1000 = 25% duty

TIM2->CCMR1 |= (6U << 4);       // PWM mode 1 on channel 1
TIM2->CCMR1 |= (1U << 3);       // preload enable
TIM2->CCER  |= (1U << 0);       // enable the output
TIM2->CR1   |= (1U << 0);       // start
```

ARR sets the frequency, CCR sets the duty

Change CCR1 at any time and the duty cycle changes on the next cycle — no glitch, because the preload register updates only at the reload point. That is what makes smooth dimming possible.

A higher PWM frequency is less visible but wastes more switching energy. For LEDs, 1 kHz is comfortably above flicker perception.

© Muním Zabidi

PWM Duty-Cycle Helper

```c
void pwm_init_tim2_ch1(uint32_t psc, uint32_t arr)
{
        RCC->APB1ENR |= (1U << 0);
        TIM2->PSC = psc;
        TIM2->ARR = arr;
        TIM2->CCR1 = 0;                 // start at 0% duty
        TIM2->CCMR1 |= (6U << 4);       // PWM mode 1, channel 1
        TIM2->CCMR1 |= (1U << 3);       // preload enable
        TIM2->CCER  |= (1U << 0);       // enable output
        TIM2->CR1   |= (1U << 0);       // start
}

// percent: 0-100
void pwm_set_duty_percent(uint8_t percent)
{
        if (percent > 100) percent = 100;
        uint32_t arr = TIM2->ARR;
        TIM2->CCR1 = ((uint32_t)percent * (arr + 1)) / 100U;
}
```

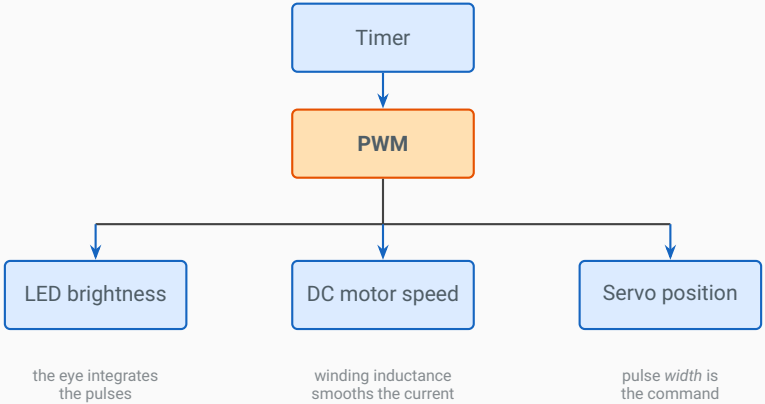
Example: A Breathing LED, Non-Blocking

```
void led_breathe_demo(void)
{
        static periodic_task_t step = { .last_ms = 0, .period_ms = 20 };
        static int duty = 0;
        static int dir  = 1;

        if (task_due(&step)) {
                duty += dir;
                if (duty >= 100 || duty <= 0) dir = -dir;
                pwm_set_duty_percent((uint8_t)duty);
        }
}
```

Every 20 ms the duty cycle steps by one percentage point up or down, driven entirely by `task_due()` — no blocking call anywhere in the loop.

One PWM Output, Three Applications



LED and motor rely on physical averaging. A servo is different: its control electronics measure the pulse width and use it to set a position.

© Munim Zabidi

PWM Output at Five Levels

Level	Representation
Physical hardware	Counter, comparator and output flip-flop driving the pad
Register	`TIM2_CCR1` compared against `TIM2_CNT`
C	`TIM2->CCR1 = 500;`
Assembly	`MOV R0, #500` / `STR R0, [R1, #0x34]`
Observable	A 1 kHz square wave at 50% duty on pin PA5

So far

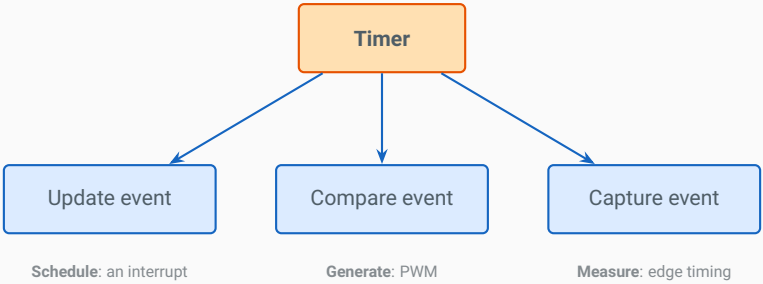
`ARR` determines the PWM period. `CCRx` determines how long the output stays high. The same two registers used earlier to schedule an interrupt now generate a waveform instead.

© Muním Zabidi

Table of Contents

1 Why This Matters
2 Why Software Delays Fail
3 Timer Architecture
4 Timing Calculations
5 A Periodic Timer Interrupt
6 Why Deterministic Timing Matters
7 SysTick: The System Time Base
8 Alternate Function Selection
9 Pulse Width Modulation
10 **Input Capture and Output Compare**
11 Engineering Case Study: Sampling Audio
12 Beamer Extras
13 Summary

Input Capture and Output Compare Three uses of one timer chain



Output compare: an event or a pin toggle when the counter reaches `CCRx`. PWM is a special case.

Input capture: when an edge arrives, the timer copies the counter into `CCRx`. The timestamp is taken in hardware, so interrupt latency does not affect it. Two captures give an interval: a frequency, a pulse width or a duty cycle.

© Muním Zabidi

Table of Contents

1 Why This Matters
2 Why Software Delays Fail
3 Timer Architecture
4 Timing Calculations
5 A Periodic Timer Interrupt
6 Why Deterministic Timing Matters
7 SysTick: The System Time Base
8 Alternate Function Selection
9 Pulse Width Modulation
10 Input Capture and Output Compare
11 **Engineering Case Study: Sampling Audio**
12 Beamer Extras
13 Summary

Case Study: Sampling Audio at 16 kHz

A keyword-spotting device listens all the time and wakes only for a word like “hey, board”. Speech needs frequencies up to about 4 kHz, so it is sampled at 16 kHz:

$$T_s = \frac{1}{16{,}000\ \text{Hz}} = 62.5\,\mu\text{s}$$

With a 1 MHz tick (PSC $= 15$):

$$\text{ARR} + 1 = \frac{1\ \text{MHz}}{16\ \text{kHz}} = 62.5$$

Not an integer. This prescaler cannot give exactly 16 kHz.

- ARR $= 61$: about 16.13 kHz
- ARR $= 62$: about 15.87 kHz
- Each is about 0.8% off.
- Exact: PSC $= 0$ (16 MHz tick) and ARR $= 999$ gives 16 kHz.

Design rule: a lower prescaler gives finer control of the period.

Why the Interval Must Be Uniform, Not Merely Frequent

A delay loop tuned to average 62.5 µs would still fail. Three properties of sampled audio explain why:

The samples define the timing

The pipeline assumes a constant interval. A late sample is not marked as late, so the waveform is time-distorted.

Jitter becomes signal error

Varying intervals add error that appears as extra spectral content. It cannot be removed afterwards.

The deadline never ends

At 16 kHz there are only 1000 cycles between samples on a 16 MHz CPU. A missed one is lost: the signal has moved on.

Budgeting the 1000 Cycles

Within 1000 cycles the system must take a sample *and* leave time for later processing. A keyword-spotting pipeline might buffer 20–30 ms of audio (320–480 samples), compute features, then run a small neural network.

Part	Its job
Timer	Sets the sampling rate.
ISR	Does the minimum: acquire the sample, store it, return.
Main loop	Does the expensive work, on a buffer the ISR has filled.

An ISR that takes longer than 62.5 µs cannot finish before the next sample is due

A hardware timer makes the sampling interval a property of the hardware, not of whatever the software happens to be doing. The same split is used at system level in Chapter 14.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

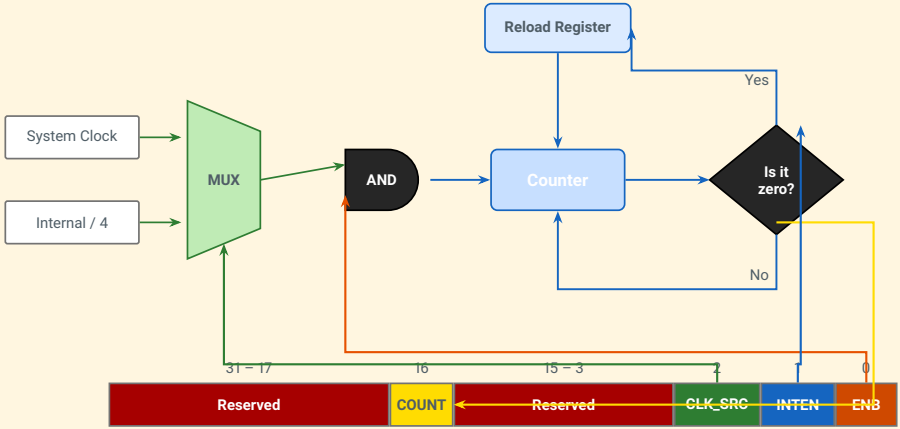
10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 **Beamer Extras**

13 Summary

SysTick, from Bits to Blocks



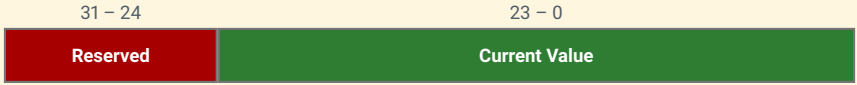
SysTick Control and Status Register

Every CTRL bit maps to one block in the datapath. The TM4C's TI bit names (`ENB`, `INTEN`, `CLK_SRC`, `COUNT`) are the same four bits CMSIS exposes as `ENABLE`, `TICKINT`, `CLKSOURCE`, `COUNTFLAG`.

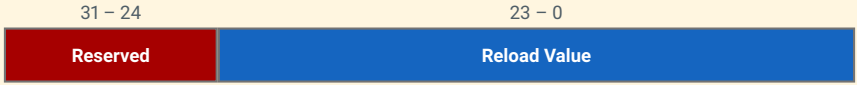
BEAMER EXTRA

Diagram adapted and redrawn from microcontrollerslab.com, "ARM Cortex-M4 SysTick timer with programming example".

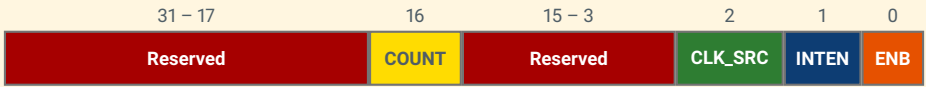
SysTick's Three Registers, Bit by Bit



SysTick Current Value Register (`VAL`)



SysTick Reload Register (`LOAD`)



SysTick Control and Status Register (`CTRL`)

Three 32-bit registers. In each one the top byte is Reserved; the `VAL`/`LOAD` halves carry the 24-bit count, and `CTRL` carries the four control bits discussed on the previous slide. CMSIS names: `SysTick->VAL`, `SysTick->LOAD`, `SysTick->CTRL`.

Register map adapted and redrawn from microcontrollerslab.com, "ARM Cortex-M4 SysTick timer with programming example".

BEAMER EXTRA

TI, ARM, and STMicro: Same SysTick, Three Register Names

SysTick is a Cortex-M core peripheral — same registers, same bits on every vendor's chip. Only the *name* the vendor's header file uses changes.

Bit	Function	TI (TM4C)	ARM / CMSIS
0	Enable the counter	`ENB`	`ENABLE`
1	Enable the SysTick interrupt	`INTEN`	`TICKINT`
2	Clock source (1=core, 0=slow)	`CLK_SRC`	`CLKSOURCE`
16	Set when counter wraps to zero	`COUNT`	`COUNTFLAG`

One implementation choice to watch

The slow clock source is implementation-defined by the Cortex-M architecture. TI's TM4C uses the system clock divided by 4; STMicro's STM32 uses HCLK divided by 8; most other Cortex-M vendors pick 8 as well.

Reading the Diagram from a Program's Point of View

Write to CTRL, left to right

- `CLK_SRC` picks the clock input to the MUX.
- `INTEN` arms the interrupt path.
- `ENB` switches on the AND gate, so the clock reaches the counter.

Three writes, in that order, is what `SysTick_Config()` does under the hood.

Read COUNT, from the top

- Counter decrements each clock.
- At zero: Yes-branch reloads from the Reload Register, and sets `COUNT`.
- Reading `COUNT` (or writing `VAL`) clears it.

The ARM TRM says this bit is sticky-until-read: do not use it for timing; use the interrupt instead.

Why this picture is worth the detour

Chapter 10 showed the SysTick API. This diagram shows the hardware behind it, so an unfamiliar vendor's SysTick code is still readable: find `ENB`, `INTEN`, `CLK_SRC`, and the reload value, and you can predict the tick rate.

Driving the Bits Directly

No CMSIS helper. Write the four CTRL bits by hand, in the order the diagram shows them — clock source, interrupt enable, counter enable — after loading `LOAD` and clearing `VAL`.

```c
#include "stm32f4xx.h"                /* STM32F446RE, Cortex-M4 */

#define SYST_CTRL_ENABLE     (1U << 0) /* bit 0  -- ENB */
#define SYST_CTRL_TICKINT    (1U << 1) /* bit 1  -- INTEN */
#define SYST_CTRL_CLKSOURCE  (1U << 2) /* bit 2  -- CLK_SRC (1 = processor clock) */

void systick_init_1ms(void) {
    SysTick->LOAD = (SystemCoreClock / 1000U) - 1U;   /* 24-bit reload */
    SysTick->VAL  = 0;                                /* clears count + COUNTFLAG */
    SysTick->CTRL = SYST_CTRL_CLKSOURCE               /* use processor clock */
                  | SYST_CTRL_TICKINT                 /* raise an interrupt  */
                  | SYST_CTRL_ENABLE;                 /* start the counter   */
}
```

The two things CMSIS does for you

`SysTick_Config()` wraps the same four writes plus an `NVIC_SetPriority(SysTick_IRQn, ...)` call, and returns `-1` if the reload value overflows 24 bits. Prefer the wrapper in portable code; use the raw form when you need a specific priority or non-default clock source.

Handler and Millisecond Delay

```c
static volatile uint32_t msTicks = 0;    /* ISR-shared, must be volatile */

void SysTick_Handler(void) {             /* fires every 1 ms */
    msTicks++;
}

void delay(uint32_t ms) {
    uint32_t start = msTicks;
    while ((msTicks - start) < ms) {
        __WFI();                         /* sleep until the next tick */
    }
}
```

Why unsigned subtraction

`msTicks - start` stays correct even after `msTicks` wraps at 2^{32}, because unsigned wrap-around subtracts cleanly. `while (msTicks < start + ms)` would fail near the wrap.

Why `__WFI()`

"Wait For Interrupt" sleeps the core until the next SysTick fires. On a battery-powered board this is the difference between a few µA and tens of mA.

Handler and `delay()` pattern adapted from Rohit Nimkar, `csrohit/bluepill-systick` (BSD 3-Clause), with `__WFI()` added. Full source: `github.com/csrohit/bluepill-systick`.

Portable: SysTick. Not Portable: Everything Around It.

Same CMSIS source runs on both boards. The lines that change are the ones that touch the vendor's silicon.

	Nucleo-F446RE	Blue Pill (STM32F103)
SysTick code — compiles unchanged		
`SysTick_Config(SystemCoreClock/1000)`	16 000 (after reset)	8 000 (after reset)
`SysTick->CTRL/LOAD/VAL`, bit names	identical	identical
`SysTick_Handler`, `__WFI()`	identical	identical
Lines that must change		
Core	Cortex-M4 (FPU, DSP)	Cortex-M3 (no FPU, no DSP)
Max clock	180 MHz	72 MHz
CMSIS header	`stm32f4xx.h`	`stm32f1xx.h`
LED pin / polarity	PA5, active-high	PC13, active-low (hw)
Port clock enable	`RCC->AHB1ENR` bit 0 (GPIOA)	`RCC->APB2ENR` bit 4 (GPIOC)
Pin mode register	`MODER` (2 bits/pin)	`CRH/CRL` (4 bits/pin)
Programmer	on-board ST-Link v2.1	external ST-Link v2 (SWD)

The pattern

SysTick is in the Arm core, so it is portable across every vendor's Cortex-M. GPIO, clocks and the LED are in the vendor's silicon, so they change on every chip.

Table of Contents

1 Why This Matters

2 Why Software Delays Fail

3 Timer Architecture

4 Timing Calculations

5 A Periodic Timer Interrupt

6 Why Deterministic Timing Matters

7 SysTick: The System Time Base

8 Alternate Function Selection

9 Pulse Width Modulation

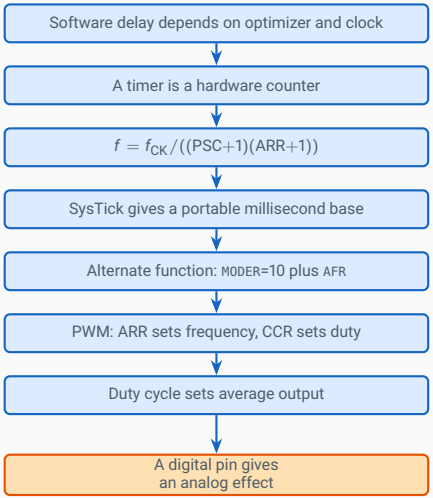
10 Input Capture and Output Compare

11 Engineering Case Study: Sampling Audio

12 Beamer Extras

13 **Summary**

Summary



Terminology covered

- Prescaler (PSC), reload (ARR)
- SysTick
- Alternate function, `AFR`
- PWM, duty cycle, `CCR`

Next: Chapter 11

ADC and Analog Interfacing

We made an analog effect. Next, we measure a real one.

© Munim Zabidi